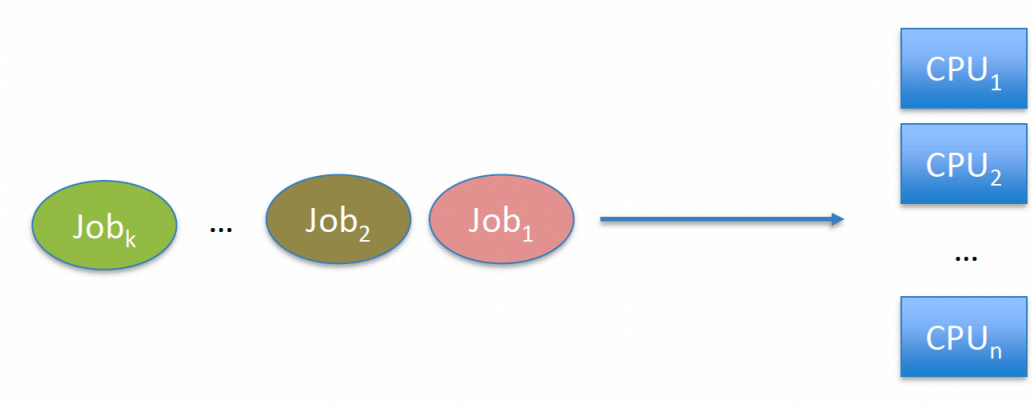
Scheduling

Description

The scheduling problem

- k jobs ready to run
- $n \geq 1$ *CPUs* that can execute them

Which jobs should we assign to which *CPU(s)*?



It selects from among the processes/threads that are ready to execute and allocates the CPU to one of them

Scheduling decisions may take place when a process:

1. Switches *from running to waiting state* (e.g., **I/O or event wait**)
2. Switches *from running to ready state* (e.g., **Interrupt**)
3. Switches *from waiting to ready* (e.g., **I/O or event completion**)
4. Terminates

- **Non-preemptive scheduling:** if the scheduling occurs only for 1 and 4 events

- **Preemptive scheduling**: the scheduling occurs in all cases

Scheduling policy: what to do next, when there are multiple threads ready to run

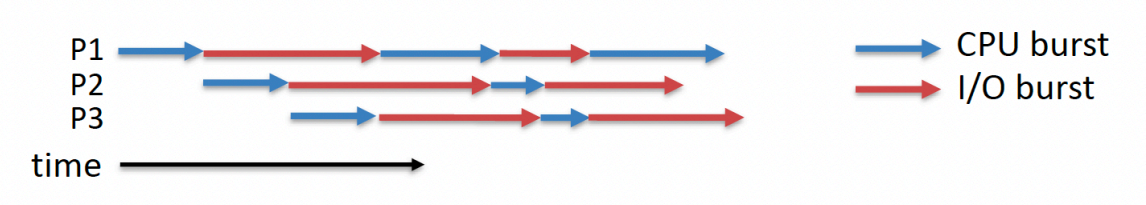
Definitions

- **Task / Job**: User request (e.g., mouse click, web request, shell command)
- **Latency / response time**: How long does a task take to complete?
- **Throughput**: How many tasks can be done per unit of time?
- **Overhead**: How much extra work is done by the scheduler?
- **Fairness**: How equal is the performance received by different users?
- **Predictability**: How consistent is the performance over time?
- **Workload**: Set of tasks for system to perform
- **Preemptive scheduler**: If we can take resources away from a running task
- **Work-conserving**:
  - *Resource is used whenever there is a task to run*
  - *For non-preemptive scheduler, work-conserving is not always better*
    - *Some systems might intentionally delay scheduling (for reasons such as energy conservation or batching tasks for efficiency) even if work is available.*
- **Scheduling algorithm**:
  - *takes a workload as input*
  - *decides which tasks to do first*
  - *performance metric (throughput, latency) as output*
  - *only work-conserving schedulers to be considered*

CPU and I/O Bursts

- **CPU Burst**: is a period during which a process is actively using the CPU
- **I/O Burst**: is a period when a process is waiting for I/O operations or events

Whena processstarts an I/O burst, anotherprocesscan use the CPU for a CPU burst:



- **CPU-bound jobs**: they have long CPU burst and a few short I/O burst
  - Scientific computation programs
- **I/O-boundjobs**: theyhavelong I/O burst and a fewshort CPU burst
  - A web browser and file transfer programs

Scheduling Criteria

- **CPU utilization**: keep CPU as busy as possible
- **Throughput**: maximize throughput (more jobs get done per unit of time)
- **Waiting time**: minimize the sum of periods a job spends waiting in the ready queue
- **Response time / Turnaround time**: minimize response time or turnaround time or both
  - ***Turnaround time***: the interval between job submission to completion (it includes all waiting time and delays)

- ***Response time***: In interactive systems, it is the time interval between the submission of a request to the first response. It does not include the time it takes to output the response

Scheduling policy goals

- **Minimize response time**, i.e., the elapsed time to do an operation or a job
  - Response time is what the user sees, elapsed time to:
    - echo a keystroke in an editor
    - compile a program,
    - execute a large problem
  - This is particularly important in interactive system

- **Maximize throughput**, i.e., the number of operations (or jobs) executed per second
  - This requires: minimize overhead (e.g., context switching), and make efficient use of system resources (CPU, disk, memory, I/O, ...)
  - Maximize throughput is particularly important in batch systems

- **Fairness**, i.e., share CPU among all users (or jobs) in some way to allow them to make progress equally

First In First Out (FIFO)

Offen referred to as **FCFS** (**First-Come First-Served**)

Schedule tasks in the order they arrive:

- Continue running them until they complete or give up the processor
- Non preemptive scheduling policy. Once a job gets the CPU, it will run until it completes or voluntarily enters in the waiting state

- Can easily be implemented using a queue of jobs

FIFO (First In, First Out) performs **poorly** on workloads where processes have **very different execution times** and arrive in a **non-optimal order**. Specifically:

When FIFO is particularly ineffective

1. **When a long process arrives first**
    - If a long job (e.g., P1 = 24t) arrives before shorter jobs (e.g., P2 = 3t, P3 = 3t), the shorter processes remain blocked until the long job finishes, causing a **high average waiting time**.
    - This issue is known as the **convoy effect**.
2. **When the arrival order is suboptimal**
    - If shorter jobs arrive after a long job, the waiting time increases significantly.
    - Example:
        - Arrival order **P1(24t), P2(3t), P3(3t)**
            - **Waiting Time**: P1 = 0, P2 = 24, P3 = 27 → **Average = 17**
            - **Turnaround Time**: (24 + 27 + 30) / 3 = **27**
        - Arrival order **P2(3t), P3(3t), P1(24t)**
            - **Waiting Time**: P2 = 0, P3 = 3, P1 = 6 → **Average = 3**
            - **Turnaround Time**: (3 + 6 + 30) / 3 = **13**
    - **FIFO is highly sensitive to the arrival order**, and a non-optimal sequence can drastically worsen performance.
3. **In interactive and time-sharing workloads**
    - In systems requiring quick responses (e.g., an OS with interactive users), FIFO can cause **unacceptable delays** for short tasks if they get stuck behind long-running processes.

Process	Arrive	Start	Stop
P1	0	0	24
P2	0	24	27
P3	0	27	30

Better alternatives

- **SJF (Shortest Job First)**: Minimizes the average waiting time by executing the shortest processes first.
- **Round Robin**: More suitable for interactive workloads, preventing starvation by time-slicing CPU access.

Shortest Job First (SJF)

Suppose to know the length of its next CPU burst. The schedule depends on the lengths to schedule the process with the shortest time

Two schemes:

- non preemptive:

  Take the task with the shortest remaining work to do and run it up to its end or until it releases the processor

- preemptive:

  Run the task with the shortest remaining amount of work to do.
  If a new shorter task wakes up: context switch and run the new task. Often called Shortest Remaining Time First (*SRTF*)

Properties of Shortest Job First (SJF)

Claim: SJF is Optimal for Average Response Time

- **Why?**
  - **SJF minimizes average turnaround time** because it executes the shortest jobs first, reducing the waiting time for subsequent jobs.
  - By executing the shortest jobs first, each job has a smaller amount of time to wait before it can complete, leading to a lower overall average response time.

For What Workloads is FIFO Optimal?

- FIFO is optimal when **all jobs have approximately the same CPU burst time** (i.e., when tasks require roughly the same amount of processing time).
  - In this case, FIFO schedules jobs in the order they arrive, which is just as effective as SJF, because there are no variations in job lengths.

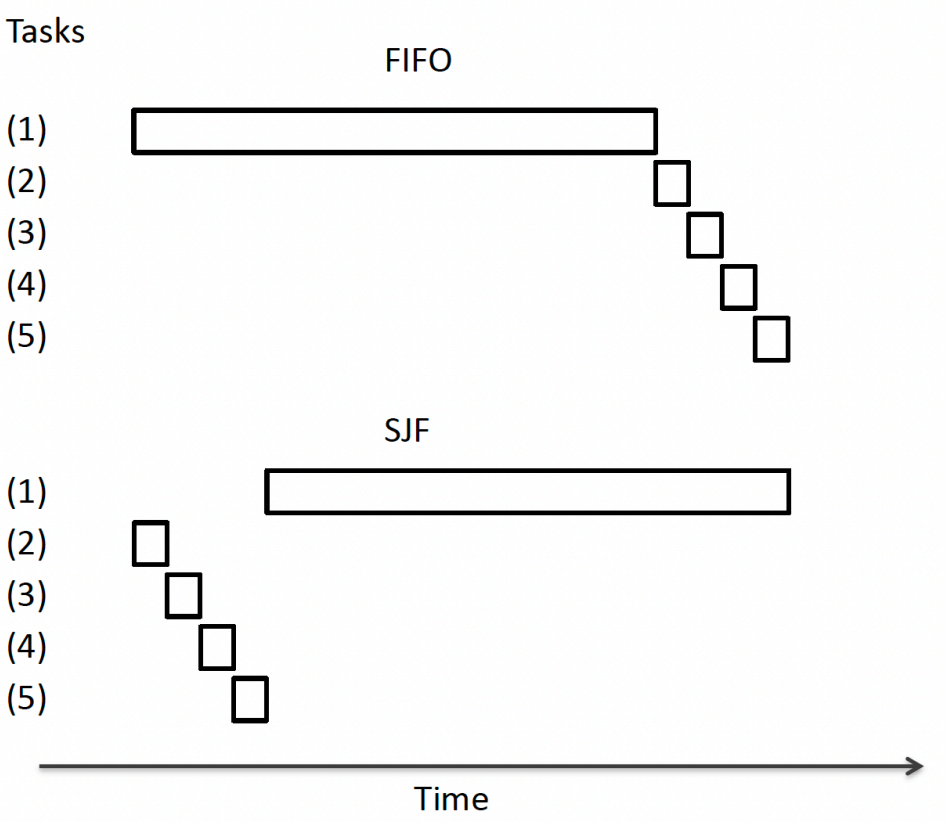
Does SJF Have Any Downsides?

- **Starvation**: Long jobs may have to wait indefinitely if there are always shorter jobs arriving.
  - For example, if there's always a new short job arriving, long jobs may never get a chance to run.
- **High Variance in Response Time**:
  - SJF can cause significant variance in response time, especially in environments where jobs can have highly different durations.
  - This is a problem for **interactive systems**, where consistency in response times is critical for user experience.

FIFO vs SJF: Comparison Between Scheduling Algorithms

Characteristic	FIFO (First In, First Out)	SJF (Shortest Job First)

Selection Criterion	Process arrival order	Process with the shortest execution time
Type	Non-preemptive	Can be both preemptive and non-preemptive
Overhead	Low (simple to implement)	Medium (requires prediction of job duration)
Average Waiting Time	High in worst-case scenarios (convoy effect)	Minimally possible (optimal for throughput)
Average Turnaround Time	Higher compared to SJF	Optimized (lower)
Efficiency with mixed workloads (long and short jobs)	Poor: short jobs may wait for a long time	Excellent: short jobs are executed first
Fairness	High (all processes are executed in arrival order)	Low: long jobs may experience starvation
Predictability	High: easy to predict	Medium: depends on job queue
Typical Usage	Batch systems, simple queue management	Optimized systems for throughput, multitasking



Example of Comparison

Scenario:

- Three processes arrive at the same time:
  - **P1: 24t**, **P2: 3t**, **P3: 3t**

FIFO (Order of arrival: P1 → P2 → P3)

- **Waiting time:** P1 = 0, P2 = 24, P3 = 27
- **Turnaround time:** P1 = 24, P2 = 27, P3 = 30
- **Average waiting time:** (0 + 24 + 27) / 3 = **17**
- **Average turnaround time:** (24 + 27 + 30) / 3 = **27**

SJF (Non-preemptive) (Order: P2 → P3 → P1)

- **Waiting time:** P2 = 0, P3 = 3, P1 = 6

- **Turnaround time:** P2 = 3, P3 = 6, P1 = 30
- **Average waiting time:** (0 + 3 + 6) / 3 = **3**
- **Average turnaround time:** (3 + 6 + 30) / 3 = **13**

Priority-based Scheduling

Overview

- In **Priority-based Scheduling**, each process is assigned a **priority** value, and the CPU scheduler selects the process with the **highest priority** from the ready queue to execute.
- **FIFO** and **SJF** can be considered special cases of priority scheduling:
  - FIFO can be viewed as priority scheduling where all jobs have the same priority.
  - SJF is a form of priority scheduling where the priority is inversely related to the burst time (shorter jobs have higher priority).

Preemptive vs Non-preemptive Priority Scheduling

- **Preemptive Priority Scheduling**: If a new process arrives with a higher priority than the currently running process, the CPU scheduler will **preempt** the currently running process and switch to the new one.
  - This ensures that higher priority tasks are executed as soon as they are ready.
- **Non-preemptive Priority Scheduling**: Once a process is running, it is not preempted, even if a higher priority process arrives.
  - The higher priority process will have to wait until the current process finishes execution.

Starvation Issue

- **Starvation** (or **indefinite blocking**) is a potential problem in priority scheduling:

- **Low-priority processes** may never get executed if there is always a higher-priority process in the system.
- This can lead to poor responsiveness for low-priority tasks, especially in systems with many short high-priority processes.

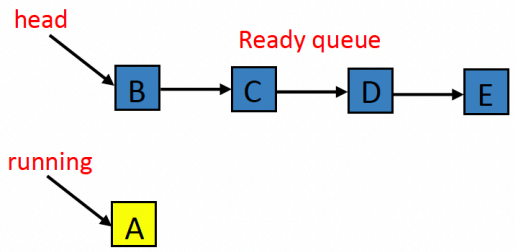
Aging Technique

- To mitigate starvation, an **aging technique** can be used:
  - Over time, the priority of processes waiting in the system for a long period is gradually **increased**, so they eventually get a chance to run.
  - This helps prevent low-priority tasks from being indefinitely blocked by higher-priority tasks.

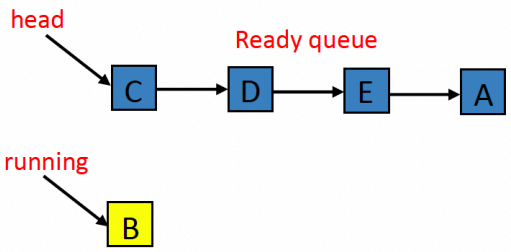
Round Robin Scheduling

Overview

- **Round Robin** is a **preemptive** CPU scheduling algorithm where each process is assigned a fixed period of time, known as a **time quantum** or **timeslice**.
- When the time quantum expires, the process is moved to the end of the **ready queue** and another process is selected to run.



(a) A in execution

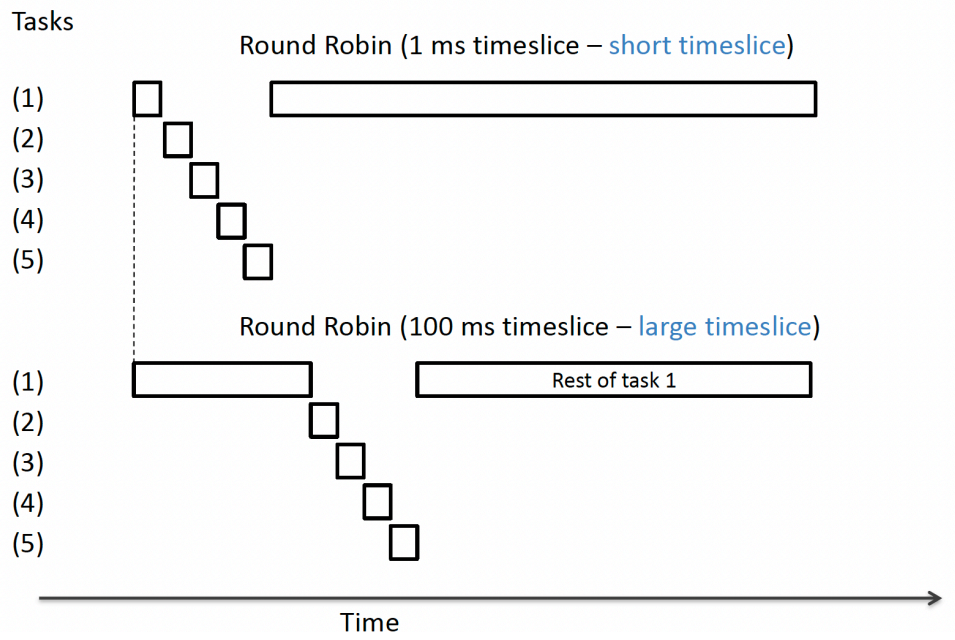


(b) A completes its time share

- **Scheduler Operation**:
  - If the process completes within the time quantum, it exits, or it may continue if not completed, being preempted and moved to the ready

queue.

- If the process completes before the timeslice is over, it is also moved back to the ready queue.
- The CPU scheduler picks the first process in the ready queue and lets it run for a timeslice. If the timeslice expires, the scheduler selects the next process.



Choosing a Time Quantum

- **Time Quantum** selection is crucial:
  - **Too long**: If the time quantum is excessively long (e.g., infinite), the system behaves like **FIFO** (First-In-First-Out), losing the benefits of Round Robin.
  - **Too short**: If the time quantum is too short (e.g., one instruction), it results in too much **context switching** overhead, spending more time switching between processes than actually executing them.
  - **Ideal range**: The typical time quantum for most modern OSs is between **20-100 milliseconds**.

Response Time

- **Response time** is the time it takes for a system to respond to a request.

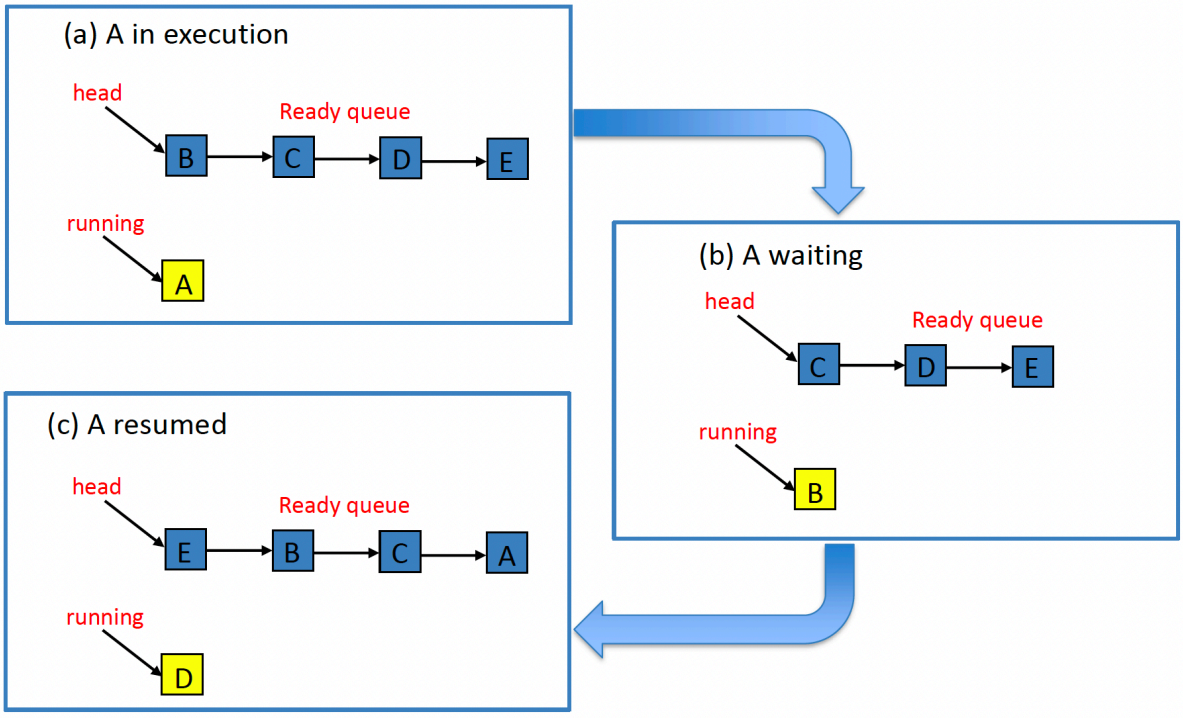
- In Round Robin, the **response time** is upper-bounded by the number of ready processes multiplied by the timeslice:

$$ResponseTime \leq (Numberof ReadyProcesses) \times (TimeSlice)$$

- This means that if there are many processes in the queue, the response time can become significant.

Management of Waiting Threads

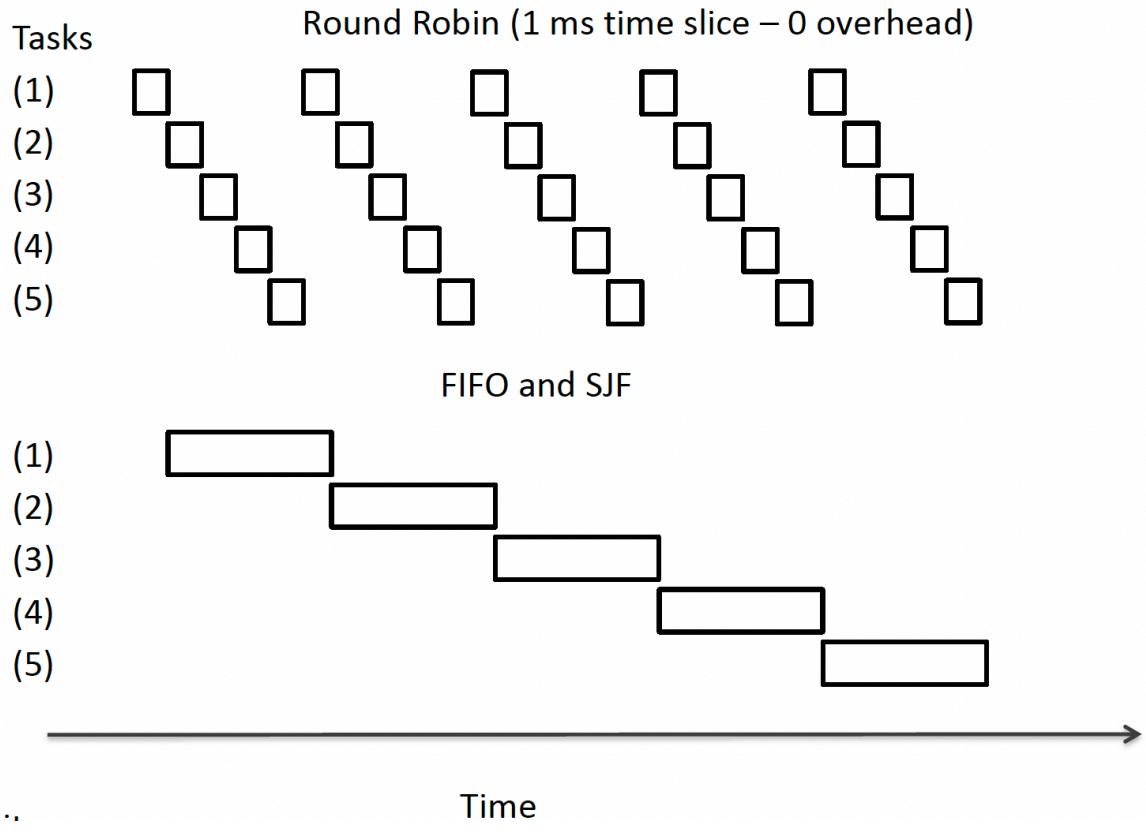
- The **timer interrupt** signals the end of a timeslice and triggers the **scheduler** to activate.
  - The scheduler will restart the timer and reschedule the tasks.
  - Even if a process is suspended (e.g., waiting for I/O), the scheduler will reassign the CPU and restart the timer.



Round Robin vs. FIFO

- **Is Round Robin always better than FIFO?**
  - **No**, Round Robin is not always superior to FIFO.

- **Reason**: Even with a zero-cost timeslice (no context-switch overhead), Round Robin can increase both the **waiting time** and **turnaround time** compared to FIFO.
- **When is this the case?** If all jobs have the same size (i.e., they take the same time to execute), Round Robin will split the jobs into unnecessary time slices, causing overhead without improving the overall execution.



Round Robin is often considered a **fair** scheduling algorithm in the sense that each process receives a fixed amount of CPU time (time quantum or time slice), but it's not always **fair** in practice, especially for **mixed workloads**. Here's why:

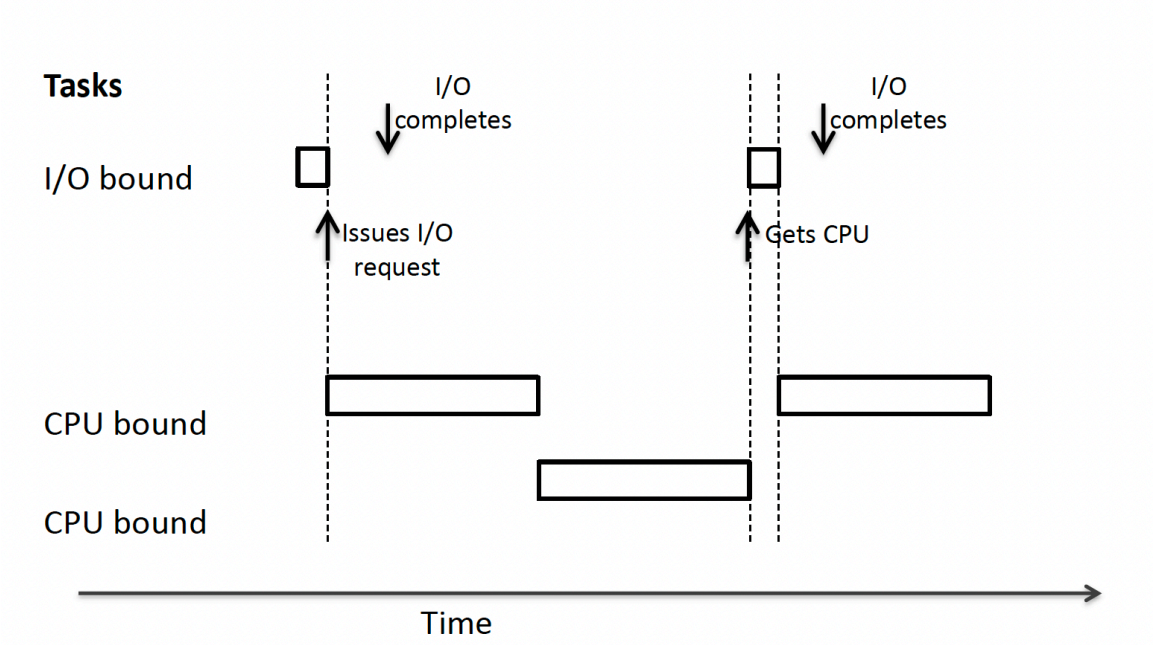
Mixed Workloads:

- **CPU-bound processes**: These processes require long CPU bursts with little or no I/O. They are computationally intensive and will use up most of their time slice without being preempted.

- **I/O-bound processes**: These processes use the CPU for short bursts and frequently perform I/O operations, meaning they spend a lot of time waiting for data or events and require less CPU time.

Why Round Robin is not always fair for mixed workloads:

1. **CPU-bound processes**:
   - In Round Robin, a CPU-bound process may have to wait for several **I/O-bound processes** to complete their small time slices, even though the CPU-bound process needs much more time to finish its work.
   - This can result in high **waiting times** for CPU-bound processes, as they are repeatedly preempted even when they could be using the CPU more effectively.
2. **I/O-bound processes**:
   - On the other hand, I/O-bound processes, which often finish their CPU burst quickly and move to I/O operations, are typically able to quickly return to the ready queue and get their small chunks of CPU time.
   - This allows them to make progress while the CPU-bound processes are stuck waiting, potentially **lowering their turnaround time**.



Fairness Breakdown in Round Robin:

- **For CPU-bound processes**: The fairness is skewed because these processes spend most of their time doing computation and are interrupted often, leading to longer wait times.
- **For I/O-bound processes**: These processes may be favored in Round Robin scheduling because their quick CPU bursts and frequent I/O operations allow them to be scheduled repeatedly with minimal overhead.

Max-Min Fairness

Max-Min Fairness is an approach to balancing a mixture of tasks, especially in environments where there are both **I/O-bound** and **compute-bound** processes.

The main goal of Max-Min Fairness is to ensure that each task gets a fair share of CPU time, especially in scenarios where some tasks require more CPU time and others require less. Here's how it works:

Key Ideas of Max-Min Fairness:

- **Balancing Resources**: The main challenge is to balance CPU time between tasks that require little CPU time (I/O-bound) and tasks that need extensive CPU time (compute-bound).
- **Maximizing the Minimum Allocation**: The idea is to maximize the **minimum share** of CPU time given to any task. This means ensuring that no task is "starved" or unfairly deprived of resources.

Approach:

1. **Identify the smallest need**:
    - If some tasks require less than an equal share of CPU time, the **smallest task** (in terms of required CPU burst) is scheduled first. This ensures that no task is left behind without a reasonable amount of CPU time.
2. **Split Remaining Time**:
    - After the smallest task is scheduled, the remaining CPU time is divided among the remaining tasks. This division is done by the **Max-Min**

approach, which ensures that the remaining tasks get a fair allocation of the remaining time.

3. **Equal Distribution**:
    - If all tasks require at least an equal share of CPU time, the remaining CPU time is split **evenly** among all tasks.

Approximation:

In a real-world system, maintaining perfect Max-Min fairness can be complex due to continuously arriving and terminating processes. A practical approximation can be made by:

- **Choosing the task with the least accumulated time**: Whenever the scheduler needs to make a decision, it selects the process that has spent the **least amount of time** on the processor. This approximation is efficient but requires maintaining order, which typically involves specialized data structures like **min-heaps** or **priority queues**. These data structures help keep track of processes' accumulated times and ensure fair scheduling.

Example:

- **I/O-bound tasks**: These tasks need only a small burst of CPU time and then typically perform I/O operations. In a Max-Min Fairness system, these tasks will receive a fair share of CPU time, avoiding starvation.
- **Compute-bound tasks**: These tasks require long CPU bursts. They can use as much CPU as they need but are still allocated CPU time in a fair manner, ensuring that they don't dominate the system and allow I/O-bound tasks to make progress as well.

Challenges:

- Maintaining Max-Min fairness in real systems requires:
    - **Efficient data structures** to track the least accumulated time.
    - **Frequent updates** to ensure that tasks with the least CPU usage get the next allocation.

Multi-Level Feedback Queue (MFQ)

Multi-Level Feedback Queue (MFQ) is a scheduling algorithm designed to balance several important goals in process management, including responsiveness, low overhead, starvation freedom, handling tasks with different priorities, and fairness among tasks with equal priority. It is widely used in systems like old versions of **Linux**, **Windows**, and **MacOS**.

Key Goals of MFQ:

1. **Responsiveness**: Ensures that tasks are given CPU time promptly, especially in interactive systems.
2. **Low Overhead**: Aims to minimize the cost of context switching and decision-making.
3. **Starvation Freedom**: Ensures that no task is left waiting indefinitely for CPU time.
4. **Priority Management**: Manages tasks with different priorities effectively.
5. **Fairness**: Ensures fairness among tasks that have the same priority level.

Key Features of MFQ:

- **Set of Round Robin Queues**: MFQ uses multiple **queues** for managing processes, each of which has a different priority.
  - **High-Priority Queues**: These queues have **shorter time slices** (quantum), meaning that processes in these queues are preempted more frequently.
  - **Low-Priority Queues**: These queues have **longer time slices**, allowing processes to run for longer periods before being preempted.
- **Scheduler Behavior**:
  - The scheduler always picks the first thread in the **highest priority queue**.
  - Inside each queue, processes are scheduled using **Round Robin**.

How It Works:

1. **High-Priority Queues**: These queues are used for tasks that need quick response times (e.g., interactive tasks). These tasks get smaller time slices, so

they are preempted more often, allowing them to quickly give up CPU time if they don't need it.

2. **Low-Priority Queues**: These are for tasks that can tolerate delays (e.g., compute-bound tasks). They get larger time slices, meaning that they are not preempted as frequently, allowing them to complete longer bursts of computation before yielding the CPU.
3. **Queue Assignment**: When a new task arrives:
    - It starts in a **high-priority queue** (if it's interactive or requires quick responsiveness).
    - If a task in a high-priority queue doesn't finish within its time slice, it may be moved to a **lower-priority queue**.
4. **Preemption and Aging**: Tasks that spend too much time in low-priority queues may be **promoted** to higher-priority queues, ensuring they eventually get a chance to execute in a reasonable amount of time.

Benefits of MFQ:

- **Responsiveness**: Interactive tasks are prioritized, ensuring quick response times.
- **Fairness**: Ensures fairness by allocating CPU time based on priority, but not letting lower-priority tasks starve.
- **Low Overhead**: Round Robin in each queue is simple and effective, with minimal context-switching cost.
- **Starvation Freedom**: By moving tasks up from low-priority queues and ensuring that no task stays at the bottom forever, starvation is avoided.

Drawbacks of MFQ:

- **Not Perfect**: MFQ aims to balance all the goals, but it does not excel at any one of them. There can still be inefficiencies or issues with fairness in certain scenarios.
- **Complexity**: Managing multiple queues with different time slices adds complexity to the scheduler, particularly in terms of maintaining the proper

queue order and priority adjustments.

Scheduling in windows

In **Windows**, the scheduler uses a **Multi-Level Feedback Queue (MFQ)** approach to manage process scheduling. The system divides processes into different priority levels, and each priority level has its own queue, much like the MFQ model described previously. Here's an example of how scheduling works in **Windows**:

Windows Scheduling Overview

Windows uses a **preemptive, priority-based scheduling** algorithm to manage processes. The scheduling is based on **priority classes** and **priority levels**. Each thread within a process is assigned a priority, and the operating system selects the thread with the highest priority that is ready to run.

Key Concepts in Windows Scheduling:

1. **Priority Classes**: Windows defines several priority classes to determine the relative importance of processes:
   - *Real-time priority class*: Used for processes that require immediate execution (e.g., multimedia applications).
   - *High priority class*: Used for critical system processes.
   - *Above normal and normal priority classes*: Used for most user applications and background processes.
   - *Below normal and idle priority classes*: Used for less critical or background tasks.
2. **Priority Levels**: Each priority class has several **priority levels** that determine how threads within that class are scheduled relative to others. Threads within higher-priority classes are always favored over threads in lower-priority classes.
3. **Quantum (Time Slice)**: Each thread is assigned a time slice (quantum), which represents how long it can run before the scheduler preempts it and gives the CPU to another thread. In Windows, time slices are dynamic and can change based on the thread's priority class and level.

How Scheduling Works in Windows (Example):

Let's consider a **real-time process**, a **high-priority user application**, and a **low-priority background process**.

- **Process 1**: A **real-time multimedia application** that requires immediate execution to maintain smooth playback.
  - **Priority class**: Real-time priority class.
  - **Priority level**: Highest priority.
  - **Quantum**: Real-time processes are assigned a **large quantum** or are never preempted until they yield or complete execution.
- **Process 2**: A **user application** performing normal tasks (e.g., web browsing).
  - **Priority class**: Normal priority class.
  - **Priority level**: Medium priority.
  - **Quantum**: User applications in normal priority have a smaller quantum, typically around **10 milliseconds**.
- **Process 3**: A **background process** that runs in the background, like a system updater or antivirus software.
  - **Priority class**: Below normal or idle priority class.
  - **Priority level**: Lower priority.
  - **Quantum**: Background processes have even smaller time slices or only get CPU time when no higher-priority threads need it.

Step-by-Step Scheduling Example:

1. **At time t0**, the system has a **real-time process** and two **user processes**. The **real-time process** has the highest priority, so the scheduler selects it for execution. It will run to completion or until it voluntarily yields.
2. **At time t1**, after the **real-time process** either finishes or yields the CPU, the **high-priority user process** is scheduled next, as it has a higher priority than the **low-priority background process**.

3. **At time t2**, the **low-priority background process** is scheduled only when there are no other **higher-priority processes** ready to run.
4. **If Process 1 (real-time)** is running and a **high-priority thread** from another application arrives, the **real-time process** is preempted (if needed, though it has the highest priority, so it's unlikely) to give the CPU to the higher-priority thread.
5. **Priority Boosting**: In some cases, Windows uses **priority boosting** for I/O-bound processes that have been waiting in the ready queue for too long. This boosts their priority to prevent starvation.
6. **Quantum Expiry**: If a process's time slice (quantum) expires and the process hasn't finished execution, it is placed at the **end of its priority queue** (for **Round Robin scheduling**). The scheduler will then move to the next highest priority process.
7. **Starvation Prevention**: Windows ensures that low-priority processes don’t starve indefinitely by using techniques like **aging** or promoting threads that have been waiting too long in the lower-priority queues to higher-priority queues, ensuring they eventually get CPU time.

Priority Inversion Problem

Priority inversion occurs when a **high-priority process** is blocked by a **lower-priority process** because the lower-priority process holds a resource that the high-priority process needs to proceed, such as a lock, semaphore, or other critical resource. This situation contradicts the expected behavior in priority-based scheduling, where high-priority tasks should not be blocked by low-priority ones.

How Does Priority Inversion Happen?

1. **High-priority process (`H`)** is waiting for a resource.
2. **Low-priority process (`L`)** currently holds the resource (e.g., a lock) that the high-priority process needs.

3. **Intermediate-priority process (`I`)** arrives and starts running, preempting the low-priority process (`L`).
4. As a result, the high-priority process (`H`) is now blocked because it is waiting for the resource held by `L`, but `L` cannot run to release the resource because it is
preempted by `I`.
5. The **high-priority process (`H`)** ends up waiting indefinitely while an intermediate-priority process (`I`) continues to run, effectively "inverting" the priorities.

In this scenario, the **high-priority process** is blocked because a **lower-priority task** is preventing access to a resource that it needs, leading to **inefficient resource usage** and **reduced system performance**.

Example of Priority Inversion:

- **Process `H` (high priority)**: Needs a lock to proceed.
- **Process `L` (low priority)**: Holds the lock, and it's performing a non-interruptible task (e.g., a long computation).
- **Process `I` (intermediate priority)**: Arrives, preempts L, and begins execution.

In this example:

- Process `H` is blocked, waiting for the lock held by Process `L`.
- Process `L` is preempted by Process I, which runs even though it's not as critical as
Process `H`.
- Process `H` cannot run, despite its higher priority, because it is waiting for Process `L` to release the lock.

Consequences of Priority Inversion:

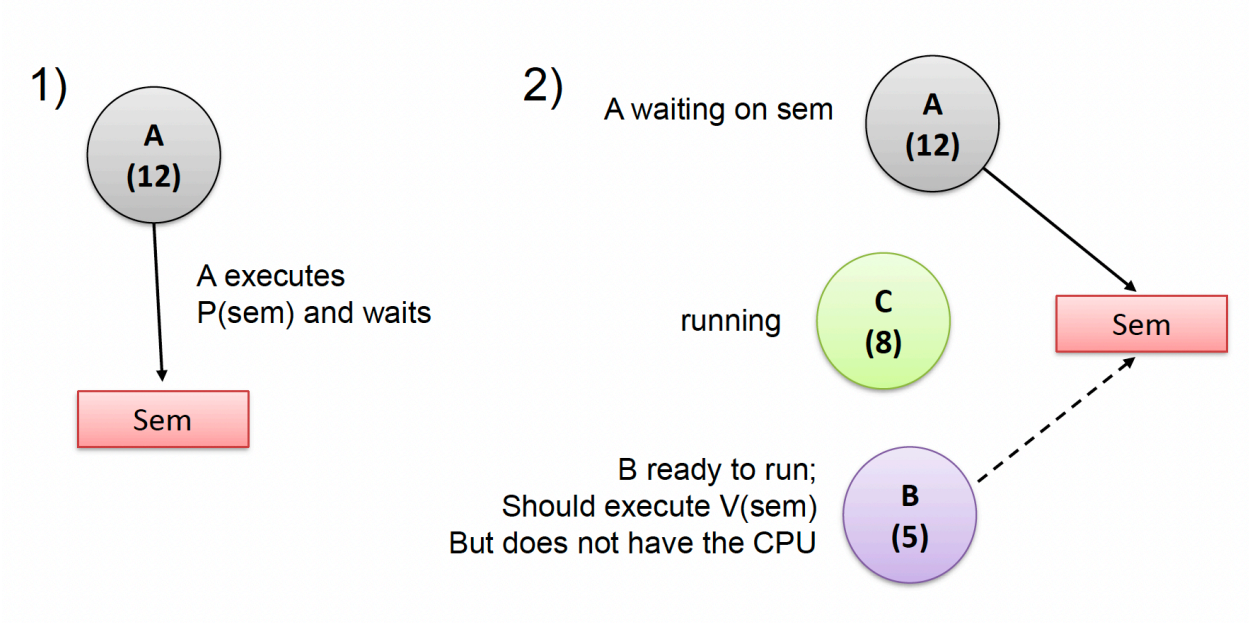
- **Unfair Resource Allocation**: Low-priority processes can block high-priority processes, which leads to unfair CPU resource distribution.
- **Starvation**: High-priority processes may experience **starvation** or be indefinitely delayed while lower-priority processes run.

- **System Inefficiency**: The system may waste CPU cycles by allowing intermediate-priority processes to run while critical tasks are delayed.

Solutions to Priority Inversion:

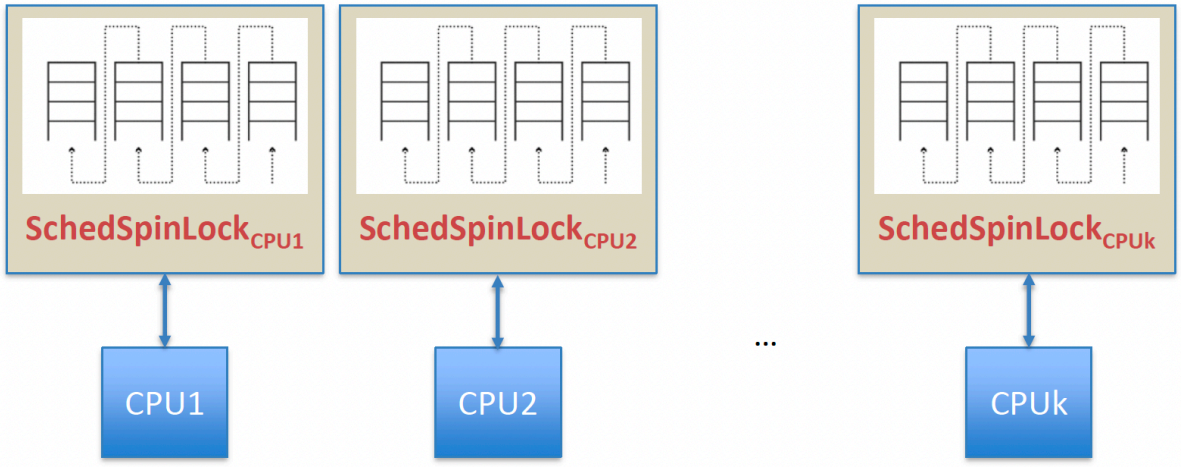
To prevent priority inversion, systems use various mechanisms such as:

1. **Priority Inheritance Protocol**:
    - When a high-priority process is blocked by a lower-priority process, the lower-priority process temporarily **inherits the priority** of the high-priority process.
    - This ensures that the low-priority process runs without being preempted by intermediate-priority tasks, allowing it to release the resource and unblock the high-priority task.
    - Once the lower-priority task finishes and releases the resource, it reverts to its original priority.
2. **Priority Ceiling Protocol**:
    - Each resource is assigned a **priority ceiling**, which is the highest priority of any task that can access that resource.
    - When a task locks a resource, it temporarily adopts the ceiling priority, ensuring that no higher-priority task can preempt it while it holds the resource.
    - This prevents priority inversion by ensuring that high-priority tasks cannot be preempted by lower-priority tasks while they hold critical resources.
3. **Locking Mechanisms**:
    - Some systems may use **atomic locks** or **timeouts** to ensure that tasks do not hold locks indefinitely, reducing the chances of a priority inversion scenario.



Multiprocessor Scheduling

One key question is how to manage multiple cores for tasks that may either be **sequential** (require one core) or **parallel** (benefit from many cores). The operating system must decide how to allocate tasks to different cores and ensure that no core is underutilized, while still maintaining fairness and efficiency.



Applying the MFQ Algorithm to a Multiprocessor System

The **Multi-level Feedback Queue (MFQ)** scheduling algorithm can be applied to **multiprocessor systems**, but it presents several challenges:

1. **MFQ on Multiprocessors**:
   - The **MFQ algorithm** generally uses a set of queues, each with a different priority, to manage processes.
   - In a **multiprocessor environment**, the scheduling needs to decide which **processor (core)** to assign to which task, in addition to managing the queues.
   - The algorithm must balance the load across multiple processors while ensuring tasks are given the appropriate CPU time based on their priority and CPU burst requirements.
2. **Challenges**:
   - **Parallel Tasks**: For highly parallel applications, we need to schedule multiple tasks simultaneously across different cores. This requires managing task affinity (which tasks should be run on which cores) and handling the **synchronization** of tasks that share data.
   - **Sequential Tasks**: Sequential tasks, which need to run on a single core, need to be scheduled efficiently without causing unnecessary contention between cores.
3. **Spin Locks vs. Plain Locks**:
   - When applying the **MFQ algorithm** or any scheduling approach to a multiprocessor, we encounter synchronization problems. A **spin lock** is often preferred over a **plain lock** for certain critical sections of the code.
   - **Why Spin Locks?**
     - **Spin locks** are used when you want to ensure that the kernel scheduler doesn't **block** while waiting for a resource. In the context of **multiprocessor scheduling**, the scheduler should not be blocked while it’s trying to allocate cores and manage queues.
     - With a **plain lock**, the thread or process will be forced to block (i.e., yield the CPU) when waiting for the lock to be released, which may lead to inefficiencies and suboptimal scheduling decisions.

- **Spin locks** avoid the blocking behavior by continuously polling for the lock, which may be less efficient in some cases but prevents the kernel from being blocked and ensures responsiveness in a real-time scheduling environment.
- In **multiprocessor systems**, where multiple cores are running concurrently, it is particularly important to avoid **blocking** the kernel scheduler since it could delay or disrupt the scheduling decisions for other processes.

Affinity Scheduling and Work-Stealing

In **multiprocessor systems**, efficient scheduling of parallel programs is essential to fully utilize available cores. One approach to achieving this is **Affinity Scheduling**, which aims to maintain cache locality and minimize overhead when scheduling tasks across multiple processors.

Affinity Scheduling

- **Per-Processor MFQs**: Each processor (or core) maintains its own **Multi-level Feedback Queue (MFQ)** for scheduling tasks. This approach helps localize scheduling decisions to each core and ensures that tasks remain on the processor where they have previously executed.
- **Spinlock Protection**: The MFQs on each processor are protected by a **per-processor spinlock** to ensure synchronization between cores while maintaining efficiency. This means only one core can modify a given processor's ready list at a time.
- **Thread Re-insertion**: When a task completes an I/O operation or is signaled by a condition variable (CV), the thread is placed back in the **ready list** of the processor on which it last ran. This helps maintain **cache affinity**, as the task will be likely to run on the same core it was previously executed on, improving performance by reducing cache misses.
- **Work-Stealing**: If a processor becomes idle, it can **steal work** from other processors that are still busy. This technique helps to **balance the workload** across processors while maintaining cache affinity. The goal is to ensure that

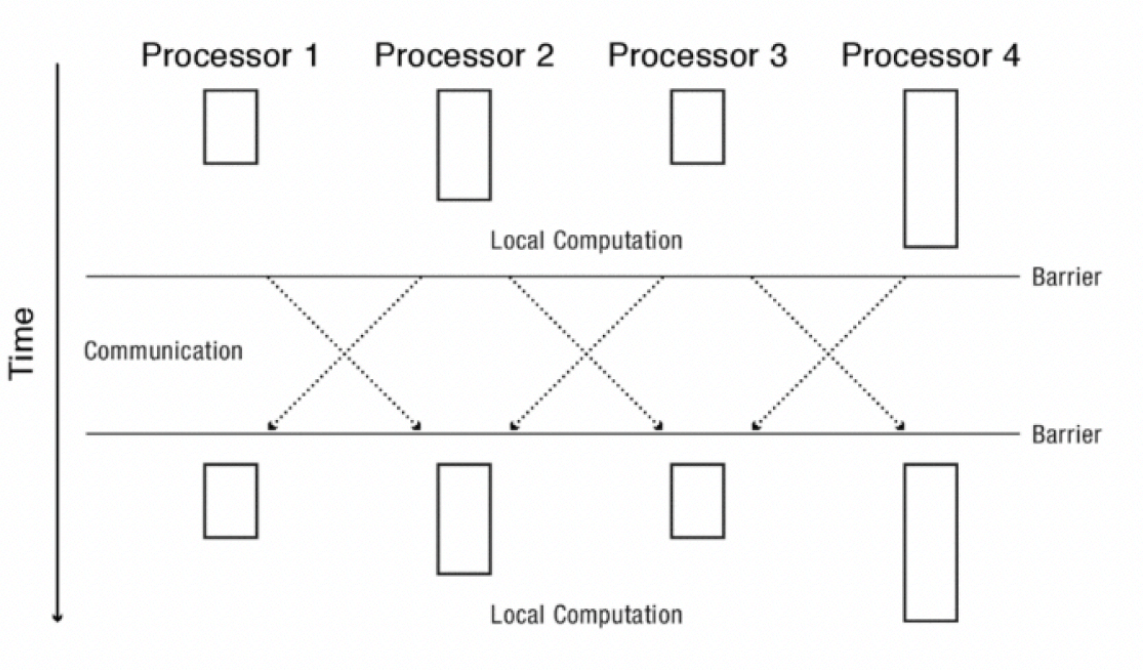
tasks are distributed in such a way that the system's overall performance is optimized while keeping cache locality intact.

Work-Stealing in Per-Processor MFQ with Affinity Scheduling

- This technique aims to achieve **load balancing** without disrupting cache locality, as tasks tend to run on the same processor where they were last scheduled.
- If a processor is idle and there are no tasks left in its ready queue, it can "steal" tasks from a processor with a higher workload. However, work stealing must be done carefully to avoid **cache misses** from tasks being executed on cores where they weren't initially scheduled.

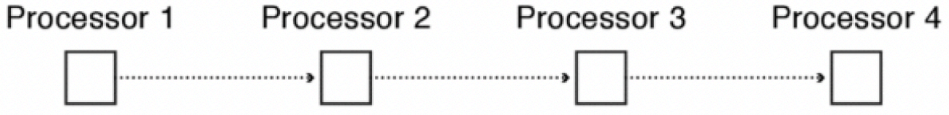
Bulk Synchronous Parallel (BSP)

- **Definition**: BSP is a parallel computing model where **K processes** (or tasks) run in a **synchronized manner**, advancing in **lockstep**. Each step, called a **superstep**, consists of two parts:
  1. *Local processing*: Each process performs its own computation or operations independently.
  2. *Global synchronization (barrier)*: After local processing, all processes synchronize at a barrier, ensuring all processes finish their tasks before moving to the next superstep.
- **Use Case**: BSP is particularly useful in problems that require synchronization and communication at regular intervals, making it ideal for algorithms that need coordinated steps across processes, like parallel graph algorithms or distributed computations.



Producer-Consumer Pipeline

- **Description**: A classic problem in parallel computing, the **producer-consumer** pattern involves two main entities:
  1. *Producers*: These processes generate data and **push it into a shared buffer**.
  2. *Consumers*: These processes **pop data** from the buffer to process it.
- **Challenges**:
  - *Synchronization*: Producers and consumers must be synchronized to ensure that consumers don't try to access the buffer when it's empty, and producers don’t overwrite data before it’s consumed.
  - *Buffer management*: The buffer must be managed to prevent overflow or underflow, and both producers and consumers need to be balanced to ensure smooth operation.

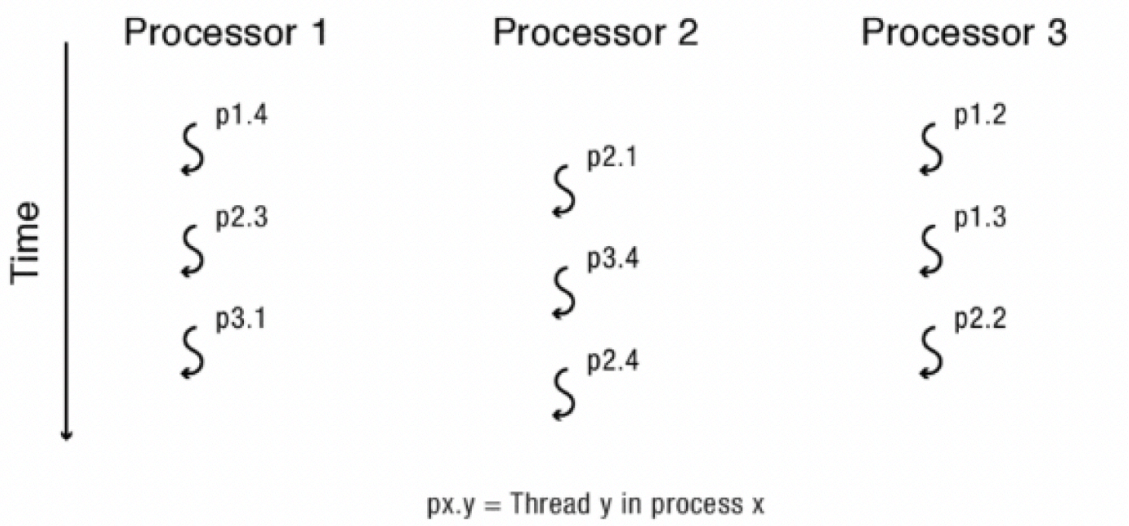


Scheduling Parallel Programs

When scheduling **parallel programs**, there are several considerations, especially when one thread is time-sliced while others from the same program are still running. While this can be handled correctly using locks and condition variables (to ensure correctness), the main concern is **performance**.

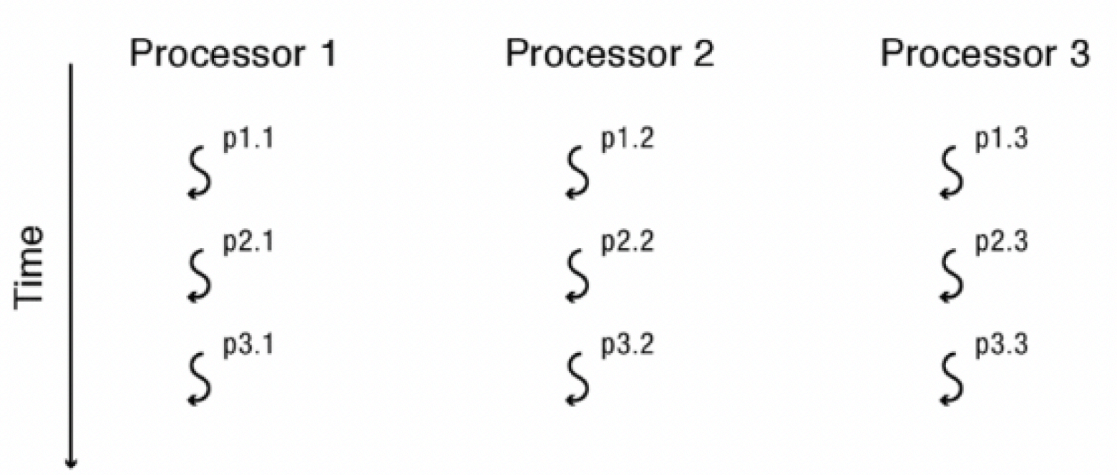
Oblivious Scheduling

- **Definition**: In **Oblivious Scheduling**, each processor independently time-slices its **ready list** without considering the state of other processors or threads. This is a simple scheduling strategy where each processor handles its own tasks independently.
- **Potential Issues**:
  - *Delay in BSP-like computations*: **Bulk Synchronous Parallel (BSP)** computations require synchronization at regular intervals, so oblivious scheduling can cause delays as processors are time-sliced independently.
  - *Delay in Producer-Consumer-like computations*: In **Producer-Consumer patterns**, the producer and consumer tasks often need to run in sync. Independent time-slicing may result in suboptimal performance or unnecessary waiting.
  - *Critical Path Delay*: Oblivious scheduling might introduce delays on the critical path of execution because it does not consider the synchronization requirements between tasks running on different processors.
  - *Preemption of Lock Holder*: If a thread holding a lock is preempted, other threads waiting for the lock may be delayed, which can cause a bottleneck and degrade performance.
  - *I/O Delay*: If some threads are performing I/O operations while others are CPU-bound, oblivious scheduling can cause delays in the system, as the I/O-bound threads might be preempted by CPU-bound threads unnecessarily.



Gang Scheduling

- *Definition*: One possible solution to the problems of oblivious scheduling is **Gang Scheduling**. In this approach, all tasks in a parallel program (or **gang**) are scheduled to run together at the same time, ensuring that no thread is time-sliced while others from the same program are still running.
- **How Gang Scheduling Works**:
  - *Synchronization*: Gang scheduling synchronizes all threads in a parallel program to ensure they run together as a "gang". This reduces the overhead and ensures that threads can proceed in lockstep, which is crucial for tasks that rely on synchronization (e.g., **Producer-Consumer**, **BSP**).
  - *Performance*: By running the threads together, gang scheduling minimizes idle time and reduces the impact of preemption on performance. It avoids critical path delays and preemption of lock holders.
- **Potential Drawbacks**:
  - *Resource Contention*: In systems with limited resources, gang scheduling might cause contention for CPU time, leading to underutilization of available cores if the program has fewer threads than the number of cores.
  - *Overhead*: The overhead of synchronizing all threads in a parallel program might outweigh the benefits in systems where tasks do not have strict synchronization requirements.



Space Sharing

- **Definition**: In **space sharing**, each parallel application or process is allocated a portion of the system's resources (e.g., CPUs or cores) and runs independently in that space. The key idea is to allocate resources such that each application or process has dedicated access to certain resources, rather than competing for them during execution.
- **Problem with Gang Scheduling**: While **gang scheduling** is effective for synchronizing parallel tasks, it can become inefficient when **multiplexing multiple parallel applications** on a system. In gang scheduling, all tasks of a program or job are scheduled to run together at the same time across multiple processors or cores.

  However, this approach doesn't scale well when different parallel applications need to run concurrently on the same system, as it could lead to idle processors or excessive synchronization overhead.
- **Efficiency Issues in Gang Scheduling for Multiple Applications**:
  1. *Resource Wastage*: If one parallel application doesn't need all the allocated processors or cores, other applications could end up waiting for resources, leading to inefficiency.
  2. *Synchronization Overhead*: The global synchronization (barrier) can become a bottleneck, especially if the applications have different processing times or resource needs.

3. *Poor Load Balancing*: The system may struggle to balance resources between multiple parallel applications that might have differing resource requirements or computational intensities.

- ***Alternative***: **Space sharing** can help resolve some of these inefficiencies by allocating fixed amounts of resources to different parallel applications, ensuring they do not interfere with each other, while also reducing the complexity of synchronization. However, the trade-off is that the system may not fully utilize all resources at all times.